

BC-TRUST: Blockchain-Assisted Trust Assessment via Auditable End-to-End Latency Evidence in SIoT

Song Luo, Tan Hu , Cong Sun , *Member, IEEE*, and Zhi Guan 

Abstract—Trust management in the Social Internet of Things (SIoT) is challenging under sparse and evolving interactions and adversarial manipulation. We present BC-TRUST, a blockchain-assisted framework that derives auditable trust evidence from on-chain protocol telemetry, avoiding ad-hoc bootstrapping and reputation-only propagation. BC-TRUST computes direct trust from inter-arrival activity trends and delay dynamics that capture both processing delay and smart-contract response delay. To preserve node privacy, BC-TRUST defines trust purely from latency and activity-based observations, and does not evaluate the semantic correctness of the reported data. It integrates recommendations via an evidence-weighted aggregation to mitigate recommendation pollution, and updates trust through a hybrid event and time-driven mechanism with offline calibration of decay parameters. We implement BC-TRUST on an Ethereum-compatible prototype and evaluate it under on-off attacks, discrimination, malicious acceleration, and recommendation manipulation. Results show stable trust evolution across windows and reduced long-term trust distortion without explicit attacker attribution, with practical off-chain overhead.

Index Terms—SIoT, blockchain, trust management, smart contract, machine learning.

I. INTRODUCTION

WITH the rapid deployment of Internet of Things (IoT) infrastructures [1], interconnected devices increasingly engage in collaborative and service-oriented interactions [2]. These interactions matter only if the parties involved can judge whether the counterpart is reliable. The Social Internet of Things (SIoT) extends conventional IoT [3] by incorporating social relationships among devices and users, which supports autonomous service discovery, cooperation, and resource sharing in heterogeneous environments [4]. Representative SIoT scenarios include smart transportation, urban infrastructure, and collaborative service platforms [5].

Trust management is a core issue in SIoT systems [6]. Unlike security mechanisms such as identity verification and access

control, trust management focuses on estimating an entity's reliability from its observed behavior over time [7], [8]. Many existing models assume that malicious behavior can be clearly detected, and then update trust values accordingly [9], [10]. In addition, purely reputation-based aggregation may amplify subjective bias and inconsistent feedback, and it can be exploited by attackers [11], [12]. As a result, SIoT systems face not only trust deficits but also privacy and security threats, including fraud, on-off behavior, and collusion attacks during service provisioning [13], [14].

In social-network trust protocols, common signals include how frequently and how recently two parties interact, as well as the duration of interactions [15]. SIoT, however, is typically more dynamic: nodes are distributed, interactions are opportunistic, and observations are often sparse. This setting calls for trust evaluation methods that remain effective under limited evidence and adversarial manipulation.

To address these challenges, this paper proposes BC-TRUST, a blockchain-assisted trust management model for SIoT. BC-TRUST evaluates trust using time-sensitive behavioral evidence derived from on-chain interaction records rather than subjective service semantics. Trust values are computed by aggregating multiple behavioral indicators, including interaction activity and delay-related signals, together with recommendation information.

Instead of directly classifying nodes as trustworthy or malicious, BC-TRUST characterizes behavioral distributions within selected time windows. An unsupervised learning method is employed to estimate system-wide behavioral tendencies, which are then used to adapt trust weighting and decay parameters. This design allows trust evaluation to remain node-oriented, subjective, and asymmetric, while avoiding explicit labeling and preserving privacy. Blockchain in BC-TRUST serves as an immutable interaction log and coordination substrate, rather than a trust oracle. Considering that interaction data in the SIoT may contain sensitive node privacy, BC-TRUST intentionally avoids inspecting semantic and data accuracy. Thus, our framework defines trust purely from latency and activity-based observations, without judging semantic correctness or data accuracy. Consequently, nodes that consistently provide incorrect but promptly delivered data would still appear trustworthy under our model.

The main contributions of this paper are summarized as follows:

- We propose a blockchain-assisted trust framework for SIoT that computes trust based on on-chain protocol telemetry,

Received 22 January 2026; revised 15 May 2026; accepted 22 June 2026. Date of publication 26 June 2026; date of current version 1 September 2026. This work was supported by National Key Research and Development Program of China under Grant 2022YFB2702505. (Corresponding author: Tan Hu.)

Song Luo is with the College of Computer Science and Engineering, Chongqing University of Technology, Chongqing 400054, China (e-mail: ratio@cqut.edu.cn).

Tan Hu is with the School of Cyber Engineering, Xidian University, Xi'an 710126, China (e-mail: neil.emola@foxmail.com).

Cong Sun is with the State Key Lab of ISN, School of Cyber Engineering, Xidian University, Xi'an 710126, China (e-mail: suncong@xidian.edu.cn).

Zhi Guan is with the National Engineering Research Center for Software Engineering, Peking University, Beijing 100871, China (e-mail: guan@pku.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3707856

such as interaction timing and delay traces, enabling auditability without relying on arbitrary initial trust.

- We design a direct-trust computation that jointly captures inter-arrival dynamics and delay dynamics, the latter comprising processing and smart-contract response delays, providing stable trust evolution under window-wise regime changes.
- We develop a time and event hybrid update and an offline calibration procedure that adapts decay parameters across context regimes, reducing on-chain exposure and computation overhead.
- We implement BC-TRUST on an Ethereum-compatible prototype and evaluate it under representative adversarial interaction patterns, demonstrating bounded trust distortion and practical overhead.

The delay evidence used in this work is the requester-observed end-to-end latency, which naturally aggregates off-chain processing time, network transmission time, and on-chain confirmation time. Our goal is not to perfectly attribute latency to a single root cause. Instead, we treat end-to-end delay as a QoS-oriented trust evidence: regardless of the underlying cause, increased latency directly degrades service usability and should therefore be reflected in the trust evaluation. To mitigate the influence of global factors such as transient congestion, we primarily interpret delay evidence within the same chain environment and within the same time window, and we apply decay mechanisms to reduce the impact of occasional fluctuations.

This paper is organized as follows: Section III introduces the system model and threat model. Section IV presents the BC-TRUST trust computation model. Section V describes prototype implementation and parameter calibration. Section VI discusses analysis and practical considerations. Section VII reports the simulation-based evaluation. Section VIII concludes the paper.

II. RELATED WORK

Prior research on IoT/SIoT trust management can be broadly categorized into the following directions.

1) *Blockchain-assisted trust management*: The development of trust management systems in blockchain has expanded to include areas such as decentralized applications (DApp) [16], cloud computing [17], wireless sensors [18], and supply chain management [19]. Deng et al. [20] propose a lightweight blockchain-based trust management scheme for resource-constrained intelligent IoT systems, aiming to make trust assessment feasible under limited computation and energy budgets. Xiang et al. [21] further develop a blockchain-enabled dynamic trust management method for the Internet of Medical Things (IoMT), emphasizing time-varying trust dynamics and resilience to behavioral changes under decentralized settings. Awasthi et al. [22] proposed a global trust scheme for P2P networks to address symmetric trust issues and mitigate various malicious attacks; however, the model does not incorporate privacy preservation. Adewuyi et al. [23] introduced CTRUST, which utilizes mathematical modeling for trust decay and dynamic contextual weights, though its trust evaluation follows a unidirectional flow without addressing privacy concerns. Gago

et al. [24] developed a framework integrating trust, privacy, and authentication to handle IoT interoperability, focusing primarily on data flows between two parties rather than network-wide transmission attacks. Boudagdigue et al. [25] modeled SIoT devices as community groups with a partially distributed leader-node structure, though its application is limited to single-factory environments. However, blockchain itself mainly offers a tamper-evident log; it does not automatically resolve trust-specific challenges such as recommendation pollution, window-wise regime shifts, or cold-start sparsity, which still require carefully designed trust evidence and update strategies.

2) *Learning-based and representation-based trust inference*: To cope with large-scale and dynamic environments, learning-based trust models have been explored to extract features and infer trustworthiness from observed behaviors. Ullah et al. [26] present DeepTrust, a deep-learning framework for dynamic trust and reputation management in IoT-based networks, focusing on data-driven trust inference under changing conditions. Jayasinghe et al. [27] employed unsupervised learning for feature categorization and SVM for trust assessment; yet, the reliance on social and system-level interaction records presents potential privacy risks. Chen et al. [28] established an adaptive protocol using honesty and collaboration indicators to resist various behavioral attacks; while effective in static scenarios, its adaptability to multi-dynamic environments could be further enhanced through machine learning integration. From a representation-learning perspective, Dhelim et al. [29] propose Trust2Vec, which leverages signed network embeddings to model large-scale IoT trust relationships and mitigate strategic reputation attacks such as self-promoting and bad-mouthing. While these approaches can improve adaptability and scalability, they typically rely on sufficient training data, model assumptions, or feature engineering, and their trust decisions may be harder to justify using auditable evidence in deployment.

3) *Gaps and motivation*: Despite these advances, two practical challenges remain prominent for highly dynamic SIoT environments: trust signals should be derived from objective and auditable evidence rather than relying primarily on subjective feedback or opaque learned representations; and the system needs a robust mechanism to calibrate trust decay under varying interaction regimes and sparse histories. In contrast to existing schemes that primarily focus on lightweight blockchain recording, dynamic trust updating, or learning-based inference in isolation, our work emphasizes a system-level trust management framework that integrates on-chain behavioral evidence, time-sensitive interaction quality, and window-wise calibration under practical deployment constraints. Our goal is not to redefine trust theory, but to provide a dependable and deployable trust management solution for SIoT environments.

III. SYSTEM MODEL & THREAT MODEL

A. System Model

We consider an SIoT environment where nodes interact through service requests and responses. Each interaction triggers a sequence of protocol events, including a service-related transaction and optional smart-contract calls for service

TABLE I
NOTATIONS AND DEFINITIONS

Symbols	Significance
PN	Public node
SN	Private node
CC	Certificate contracts written by smart contracts
PCC	Public node's certificate contract
SCC	Private node's certificate contract
TPN	Global trust values for public nodes
TSN	Global trust values for private nodes
TCN	Trusted computing node
DT	Direct trust value
RT	Recommended trust value
IA	Interaction activity
PD	processing delay
RD	Smart contract response delay
α	decay regime coefficient
β	Interaction activity weights
γ	Processing delay weights
μ	Smart contract response delay weights
λ	DT-RT mixing weight

registration, discovery, and auditing. The blockchain provides an append-only, totally ordered log of these events. BC-TRUST extracts protocol telemetry from this log, including timestamps, inter-arrival intervals, and delay components comprising both processing and smart-contract response delays. Trust is computed per node over discrete windows of length W , and the resulting deviation score is used by consumers to rank and select service providers.

B. Threat Model

We assume the blockchain ledger is tamper-resistant as long as the adversary does not violate the underlying consensus security threshold. For uncompromised identities, we assume standard cryptographic security providing signature unforgeability; if a private key is stolen, the corresponding identity is considered compromised and out of scope. We consider an adaptive adversary who can control a subset of nodes. While the adversary may attempt to create Sybil identities, doing so is heavily subject to deployment-specific resource constraints. Therefore, we also consider the following alternative attack strategies where the adversary leverages a limited number of identities: interaction manipulation, where the adversary generates arbitrary interaction schedules to inflate trust within a window; on-off behavior, where malicious nodes alternate between benign and malicious behaviors across windows; discrimination, where a node selectively degrades service for specific victims while behaving well to others; and recommendation manipulation, including bad-mouthing and ballot-stuffing. Our goal is trust correctness under observable on-chain records, i.e., limiting sustained trust distortion and maintaining stable trust evolution, rather than guaranteeing availability under network-level DoS.

IV. BLOCKCHAIN TRUST MANAGEMENT MODEL

Trust is defined as protocol-level behavioral reliability derived from observable interaction records, rather than semantic service quality or subjective trust perception. Table I presents the essential parameter notations and explanations used in this paper. Our proposed model is particularly suitable for

SIIOT scenarios involving interactions between devices and human-operated endpoints, insofar as such interactions are mediated and recorded through blockchain smart contracts. Thus, these interactions and operations are recorded on the blockchain. We structure the next section as follows: First, we will introduce the model's framework and examine the requirements of the SIIOT trust management system. Second, we will present the trust management algorithm's details, including the global trust value, interaction activity level, processing delay, smart contract response delay, and recommended trust value.

A. System Architecture

Distributed throughout the network system, the devices within the SIIOT platform collaborate to provide and select optimal services. The convergence of the trust management model is vital for improved functionality, whereby characteristics of SIIOT and trust management combine. As a result, this paper focuses on the importance of design ideas for trust management models suitable for SIIOT. Based on work done in Chen et al. [28] rely on subjective recommendations and time-decay. BC-TRUST replaces these with objective blockchain telemetry to eliminate manipulation risks such as IA, PD, and RD. Awan et al. [30] model typical IoT threats using traditional feedback. BC-TRUST enhances resilience via a behavioral deviation score and unsupervised calibration, suppressing long-term manipulation without explicit attacker labeling. Kouicem et al. [31] use blockchain as a decentralized database for trust scores. BC-TRUST extends this by leveraging underlying operational attributes such as block timestamps and contract invocations as zero-preset evidence sources. Sharma et al. [32] identify vulnerabilities in reputation systems under dynamic environments. BC-TRUST addresses these gaps through a window-based dynamic calibration framework that captures behavioral shifts under practical deployment constraints.

We follow the system and threat model assumptions stated in Section III. For completeness, this section focuses on the trust computation pipeline and on-chain evidence organization.

In this SIIOT, the PNs function as information disseminators and can acquire knowledge about the service categories offered by the SIIOT group through PCCs. The group offers various service categories, and SNs are service nodes within this group that provide corresponding services through SCCs. For example, in the online trading platform scenario, merchant A provides three independent service resources and sets up three departments, respectively. At this time, the management PN in merchant A is required to compile the PCC, and the three departments are the SNs to provide the SCCs of the corresponding data resources, respectively. This enables devices belonging to independent merchants, enterprises, or individuals in the SIIOT to obtain trust evaluations under the proposed model. This ensures that IoT devices are interoperable.

As shown in Fig. 1, the architecture of BC-TRUST consists of four layers, including the Internet of Things Device Layer(IOTL), the Access Forwarding Layer(AFL), the Trust Management Layer(TML), and the Blockchain Service Layer(BCL).

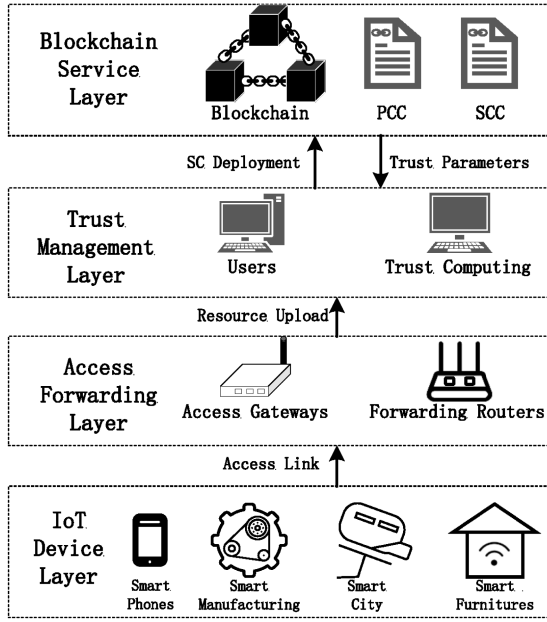


Fig. 1. BC-TRUST architecture.

B. A Trust Management Model for SIoT

This study adopts Gambetta's seminal definition of trust [33], which frames trust as a subjective probability regarding an agent's future actions within a decision-making context. While this captures the sociological essence of trust, our model operationalizes this concept by relying exclusively on objective, protocol-level behavioral evidence recorded on the blockchain. We deliberately avoid inferring latent intentions or subjective beliefs. Furthermore, the framework does not equate low interaction frequency with malevolence. Instead, IA is synthesized with delay patterns and temporal consistency; this multi-dimensional approach ensures that sporadic yet high-quality interactions are not penalized, maintaining a neutral stance toward dormant but honest nodes.

Definition 1 (Global Trust): The trust value of node x within the blockchain system is determined by its interaction with other nodes during a specified period. We calculate this value using both DT and RT. The overall trust value of node x is denoted as $TPN(x)$ for the PN and $TSN(x)$ for the SN.

$$TPN(x) = \frac{\sum_{i=1}^n w_i \cdot TSN(i)}{\sum_{i=1}^n w_i}, \quad w_i = |\mathcal{E}_{x,i}|, \quad (1)$$

$$TSN(x) = \lambda \times DT + (1 - \lambda) \times RT, \quad (2)$$

Where n represents the number of sets of SNs for which the PN is responsible and $\lambda \in [0, 1]$. λ is derived from aggregated system behavior and does not introduce feedback from global statistics to individual behavioral signals. w_i denotes the amount of on-chain evidence between PN x and its i -th SN within the selected time window, i.e., the number of interaction records $\mathcal{E}_{x,i}$. This normalization prevents a PN from inflating $TPN(x)$ by simply attaching additional SN identities without interactions: SNs with no evidence contribute $w_i = 0$ and do not change the aggregated trust. If $\sum_i w_i = 0$, the PN is treated

as having insufficient evidence in the current window and the global aggregation is not updated for that window. TPN represents an aggregated trust indicator derived from the associated TSNs within a given SIoT group, rather than an absolute or system-wide trust authority.

We define the deviation score S as a zero-centered behavioral deviation score $S \in \mathbb{R}$. Scores closer to 0 indicate behavior consistent with the normal regime, while larger magnitude $|S|$ indicates stronger deviation and higher abnormality risk. The *sign* of S captures the direction of temporal drift within the window: $S > 0$ indicates recent improvement (more responsive near the end), whereas $S < 0$ indicates recent degradation (less responsive near the end). Importantly, S is a deviation score. Therefore, the trust decision is made by testing whether $|S|$ stays within a tolerance band around 0 (i.e., $|S| \leq \tau$). $TSN(x)$ is the aggregated trust value, while S is the zero-centered deviation indicator derived from windowed evidence and used for acceptance testing. Concretely, we compute S by normalizing TSN within each window, so that S is comparable across windows and centered at 0.

We standardize trust within window w by

$$S_w(x) = \frac{TSN_w(x) - \text{median}(TSN_w)}{\text{MAD}(TSN_w) + \varepsilon}, \quad (3)$$

where $\text{MAD}(TSN_w) = \text{median}(|TSN_w - \text{median}(TSN_w)|)$ and ε avoids division by zero. This formula uses the median and Median Absolute Deviation (MAD) rather than the traditional mean and standard deviation to resist outlier manipulation. In SIoT environments, attackers can inject extreme interaction delays to distort the global trust baseline. The median and MAD are robust against such extreme outliers, ensuring that a minority of malicious nodes cannot distort the evaluation of normal behavior.

C. Direct Trust Value

Behavioral signals are not trust values themselves, but observable indicators used for trust aggregation. DT is an aggregation of behavioral signals, not a direct anomaly score.

Definition 2 (Direct Trust): The blockchain can store comprehensive records of past interactions between objects, with the DT of these objects being ascertainable through limited and privacy-preserving data. It comprises interaction activity (IA), processing delay (PD), and smart contract response delay (RD). It is important to note that the system-wide behavioral context is not assumed to be known at runtime. It is important to clarify that PD measures the relative temporal deviation rather than the absolute processing latency. In a highly heterogeneous SIoT environment, absolute processing capabilities vary drastically across devices. Evaluating absolute latency would unfairly penalize resource-constrained devices. By recursively calculating the variance against the node's own historical baseline, PD accurately isolates malicious performance degradation from inherent hardware limitations. Instead, α is a window-level context heterogeneity index computed from system-wide on-chain telemetry without any malicious labeling. We use α only to select the decay regime in (4); it is not used for online attacker

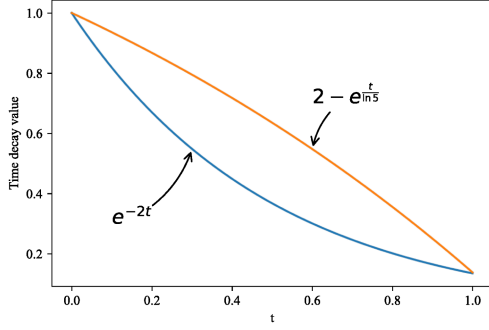


Fig. 2. Two trust value decay functions.

attribution.

$$\begin{cases} (2 - e^{\frac{t}{0.5}}) \times (\beta \times IA + \gamma \times PD + \mu \times RD), \alpha < 0.7 \\ e^{-2t} \times (\beta \times IA + \gamma \times PD + \mu \times RD), \alpha \geq 0.7, \end{cases} \quad (4)$$

Where $\beta + \gamma + \mu = 1$ and t denotes the relative time normalized to $t \in [0, 1]$. Fig. 2 illustrates two temporal decay regimes in (4) controlled by the context heterogeneity coefficient α . We interpret α as a global risk indicator derived from the system-wide trust assessment within a window, i.e., a data-driven heterogeneity index reflecting how dispersed the system-wide telemetry is within the current window; thus, we adopt a faster-decaying update so that outdated “good history” is forgotten quickly and recent misbehavior dominates. When the estimated malicious prevalence is low, we adopt a slower decay to preserve stable reputations and reduce unnecessary volatility. In our experiments, we use 0.7 as a practical operating point to switch between these two regimes, i.e., if the estimated benign proportion falls below 0.7, we activate the fast-decay policy; otherwise, we use the slow-decay policy. This design reflects a worst-case-aware stance under adversarial environments while remaining stable under benign conditions. Offline historical windows are used only to tune hyperparameters (e.g., DBSCAN eps/minPts and the regime threshold), while α itself is computed per window at runtime.

In (4), all three metrics are normalized such that positive values indicate benign, responsive behavior, while negative values indicate malicious, delayed, or degraded behavior. For instance, severe processing delays cause the current interval e_m to significantly exceed the historical average AD , pushing PD and similarly RD into the negative range. Crucially, PD measures this relative deviation rather than absolute latency, which isolates malicious degradation from inherent hardware limitations and prevents unfairly penalizing resource-constrained devices. To guarantee that worse delays always penalize trust, the weighting coefficients must be strictly positive fractions, satisfying $\beta, \gamma, \mu \in (0, 1)$ and $\beta + \gamma + \mu = 1$. Consequently, the linear combination $(\beta \cdot IA + \gamma \cdot PD + \mu \cdot RD)$ ensures that any behavioral degradation strictly and monotonically reduces the aggregated DT value, prior to the application of the time-decay function.

To avoid compromising node privacy, we rely only on blockchain-observable protocol telemetry to infer whether a

window exhibits abnormal delay patterns, rather than inspecting service content or semantics. Therefore, the direct trust value serves as a temporal behavioral assessment derived from blockchain time intervals, rather than a semantic evaluation of service quality.

The blockchain records block time as timestamps, which provide a natural notion of physical time. However, in public blockchains the inter-block time is not strictly constant and may fluctuate due to network and consensus dynamics. Importantly, BC-TRUST does not require a perfectly uniform clock; it only requires a monotonically increasing ordering of interaction records within an observation window. Therefore, time intervals in our behavioral indicators can be computed either in physical time using block timestamps, or in logical time using the ordering index of interaction records. In our Ganache-based prototype, blocks are generated under a controlled setting, which makes the logical-time approximation convenient for implementation, while the formulation remains applicable when using timestamps in real deployments.

1) *Interaction Activity Algorithm*: The polynomial fitting is adopted as a heuristic approximation to capture interaction continuity trends, rather than claiming an optimal or unique modeling choice.

Definition 3 (Interaction Activity): The nodes in recent operations are either active or inactive. We analyze any number of recent on-chain transactions from the specified node to determine its current willingness to interact.

As per the most recent blockchain block, select the latest interactions and block N that node u possesses a history in the blockchain for, labeled as

$$O_p = \{o_1, o_2, \dots, o_n\} \quad (5)$$

So the interaction interval can be expressed as

$$\begin{aligned} I &= \{N - o_n, o_n - o_{n-1}, \dots, o_2 - o_1\} \\ &= \{i_1, i_2, \dots, i_n\}. \end{aligned} \quad (6)$$

To manifest the positive and negative aspects of the intervals, each value in I will subtract the average value of I , denoted as

$$\begin{aligned} M &= \left\{ i_1 - \frac{\sum_{k=1}^n i_k}{n}, i_2 - \frac{\sum_{k=1}^n i_k}{n}, \dots, i_n - \frac{\sum_{k=1}^n i_k}{n} \right\} \\ &= \{m_1, m_2, \dots, m_n\}. \end{aligned} \quad (7)$$

The least squares method is adopted for parameter estimation due to its computational efficiency and robust convergence [34]. Specifically, we utilize the Matlab polyfit function [35] to perform linear regression, deriving polynomial coefficients that characterize the relationship between independent and dependent variables. To facilitate the direct trust calculation, the definite integral is negated so that positive feedback is represented by positive values. The formula is as follows:

$$IA = - \int_a^b f(x) dx. \quad (8)$$

Considering the noisy and constrained nature of on-chain observables, we develop Interaction Activity (IA) as a lightweight

summarizer to map windowed interaction patterns into comparable scalar values. This design facilitates unified ranking and binary classification. Polynomial smoothing is integrated to mitigate sporadic jitter, yielding a compact metric suitable for downstream decision-making. While not claimed as statistically optimal, this estimator prioritizes computability, repeatability, and comparability within on-chain environments. To handle sparse or bursty data, we implement a minimum-observation constraint as a deliberate engineering trade-off.

2) Historical processing delay:

Definition 4 (Processing Delay): Anomalous intervals within the selected observation window are used to characterize the interaction behavior of the target node. A positive result confirms the absence of delay, while a negative result indicates the presence of delay.

Focusing on user-initiated delays rather than system-level latencies (e.g., network or mining), we model the temporal intervals between a node's consecutive blockchain interactions. By chronologically sequencing these events, we derive a behavioral indicator that characterizes temporal consistency. This metric reflects relative deviation patterns rather than an absolute measure of trustworthiness. Let the interaction set be $O_p = \{o_1, o_2, \dots, o_n\}$. Similar to (6), its corresponding interaction interval set is $I_e = \{e_1, e_2, \dots, e_m\}$. We calculate the mean of all interaction intervals in I_e , excluding the last interval, to obtain the average delay (AD), denoted as follows:

$$AD = \frac{1}{m-1} \sum_{j=1}^{m-1} e_j, \quad (m > 1). \quad (9)$$

We then compare AD with the previous interaction interval e_m to obtain the processing delay (PD_{new}) of the most recent interaction interval during that period. It will be denoted as follows

$$PD = \frac{AD - e_m}{AD}. \quad (10)$$

To determine whether node p is active in this period, we run the processing-delay algorithm over all interaction intervals to obtain the historical processing delays (HPD) for that period. To do so, we process all historical intervals in the same period according to the following Algorithm 1:

We summarize the period-level delay tendency by $HPD_{\text{sum}} = \sum_k HPD[k]$, where positive values indicate no abnormal delay trend and negative values indicate persistent delay relative to the historical baseline in the window. The algorithm recursively invokes the processing delay algorithm. The time complexity is $O(n)$. It sequentially retrieves the historical processing delay of node p for the relevant period. It also removes the most recent interaction interval for each processing delay calculation, thereby reducing storage and computational overhead. The sum of the HPDs represents the PD, and when the result is greater than 0, node p is deemed positive for that period; when the result is less than 0, indicates that node p exhibits a persistent delay tendency.

3) Smart contract response delay:

Definition 5 (Smart Contract Response Delay): Within a selected observation window, we analyze the ordered smart

Algorithm 1. Historical Delay Algorithm.

```

input : Array  $I_e = [e_1, e_2, \dots, e_m]$ 
output: Array  $HPD$  (historical processing delays)

1 if  $\text{length}(I_e) == 0$  then
2    $HPD \leftarrow []$ ;
3   return  $HPD$ ;
4 end
5 if  $\text{length}(I_e) == 1$  then
6    $HPD \leftarrow []$ ;
7   return  $HPD$ ;
8 end
9 else
10   $\text{IntervalPop} \leftarrow I_e.\text{pop}()$ ;
11   $n \leftarrow \text{length}(I_e)$ ;
12  if  $n == 0$  then
13     $HPD \leftarrow []$ ;
14    return  $HPD$ ;
15  end
16   $AD \leftarrow \frac{1}{n} \sum_{j=1}^n e_j$ ;
17   $PD \leftarrow \frac{AD - \text{IntervalPop}}{AD}$ ;
18   $HPD \leftarrow \text{HistoricalProcessingDelay}(I_e)$ ;
19  append  $PD$  to  $HPD$ ;
20  return  $HPD$ ;
21 end

```

contract invocation records issued by node q . Smart contract response delay captures whether node q exhibits selective latency when invoking different functions under comparable conditions. Such function-selective delay may indicate service degradation or function-level discrimination such as biased treatment across service functions, rather than a proof of intent.

We decompose observed latency into PD and RD. PD reflects the service-side processing quality and resource contention, while RD reflects the responsiveness of contract-mediated operations, such as registration, auditing, or payment settlement. This separation is critical in SIoT because a node may behave well at the application layer but degrade contract-related operations, and such behavior should be penalized differently. We therefore treat PD and RD as distinct trust evidence and combine them with calibrated weights.

During the designated period, we obtain all smart contract interaction processing records for node q by traversing its set $O_q = \{o_1, o_2, \dots, o_n\}$, yielding the interaction interval set $I_s = \{s_1, s_2, \dots, s_{n-1}\}$.

We should note that both the processing delay and the smart contract response delay utilize a standard algorithm. In practice, a block may contain multiple interaction transactions and contract calls. BC-TRUST treats each interaction as an individual on-chain record, ordered by the tuple $(\text{blockNumber}, \text{txIndex})$. When a scalar index is needed, we map the tuple into a strictly increasing logical index and compute intervals on this ordered sequence. This removes the need for the simplifying assumption of “one interaction per block” while keeping the same interval-based computation pipeline. Fig. 3 illustrates the contrast between the two, where the blockchain

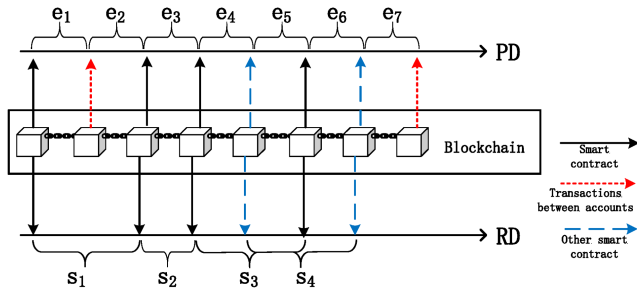


Fig. 3. PD and RD different ways of taking interval values.

fragment includes the response of the same node to two distinct smart contracts and a virtual coin transaction to another node. The processing delay depends on the sequential acquisition of block numbers. In contrast, RD is computed from request-response transaction pairs linked by an event identifier; it measures the block gap between the request and the corresponding on-chain response.

D. Recommended Trust

We do not eliminate subjective feedback; instead, we treat recommendations as auditable on-chain records and explicitly constrain their influence using direct evidence to mitigate pollution.

Definition 6 (Recommended Trust Value): We calculate the RT value for recommendation trust by collecting service quality evaluations from interaction objects. We derive this data from other nodes' ratings of the CC, using a positive integer within a score value range of 0 to 5.

Retrieve all evaluations of CC established by a service node on the entire blockchain. This information contains the evaluation block number, the commenter's account address, and the evaluation score. As a commenter can revise the evaluation of a CC by commenting again, only the latest comment score from that commenter is obtained.

The algorithm generates a list comprising evaluation attenuation factors (EAF). The time complexity is $O(n)$. These factors correspond successively to the evaluation information provided by other nodes in the CC system. We can calculate the influence of each comment score by multiplying the EAF, which represents the weights of the comment score, sequentially with each comment score.

BC-TRUST does not enforce a single global notion of trust. Instead, each node derives trust values based on its own parameterization and interaction perspective.

V. PROTOTYPE IMPLEMENTATION AND PARAMETER CONFIGURATION

This section outlines our experimental setup for validating the feasibility and reproducibility of BC-TRUST. Rather than optimizing performance, we demonstrate deployability on a real blockchain and the practicality of extracting behavioral signals and configuring trust parameters. We use two complementary settings: an on-chain Ganache prototype to verify smart-contract functions and interaction logging, and an off-chain replay that

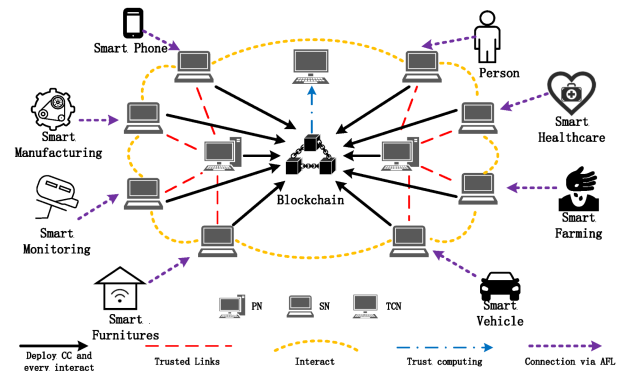


Fig. 4. BC-TRUST model.

converts logs into arrays for large-scale trust computation and overhead evaluation on the same hardware.

A. Experimental Tools

We implement the prototype on an Ethereum-compatible [36] private chain (Ganache [37]) and perform off-chain log [38] replay for trust computation.

In our Ganache-based prototype, the ten virtual account nodes consist of two PNs and eight SNs. These nodes are simulated as lightweight local RPC accounts rather than fully isolated hardware virtual machines. Each virtual node is assigned a distinct ECDSA key pair and an initial balance of 100 simulated ETH to cover smart contract gas fees. Since they operate on a unified local testnet, all virtual nodes share the computational resources of the underlying host machine, which is equipped with an 11th Gen Intel Core i5-11260H processor. This specific setup is designed to validate the smart contract logic, on-chain evidence logging, and gas cost feasibility under standard Ethereum protocols. Through these nodes, we implemented the BC-TRUST model on Ethereum, our chosen blockchain platform. A prototype system was deployed for evaluation using the proposed BC-TRUST model, as shown in Fig. 4. This Ganache setup is used to demonstrate feasibility and reproducibility of the end-to-end pipeline; larger-scale statistics reported later correspond to off-chain processing of extracted interaction records rather than increasing the number of Ganache accounts.

Since authentication information needs to be stored in the CC we designed, we assume that the PN can be authenticated by external identity mechanisms independent of the blockchain. This assumption is orthogonal to the proposed trust computation model, which operates solely on observable on-chain interaction records.

B. Smart Contract Details

In BC-TRUST, smart contracts provide an immutable, auditable log of interaction records and manage trust-link membership via certificate contracts (CCs). A PN-side certificate contract (PCC) maintains the set of linked SNs and exposes interfaces for membership updates and queries, while an SN-side certificate contract (SCC) supports submitting evaluation feedback. We implemented a prototype contract to register PN/SN

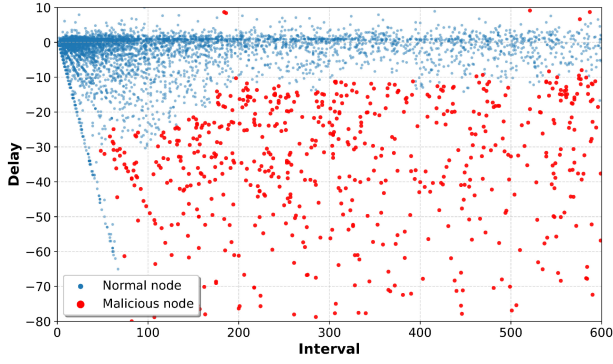


Fig. 5. Results of the dataset after DBSCAN clustering.

identities and their CC addresses, record the evidence required for trust computation, and support recommendation submission under role-based access control. Membership removal uses a standard swap-with-last and pop pattern and deletes the corresponding mapping entry.

C. Parameter Configuration

In BC-TRUST, machine learning is not used to infer trust-worthiness or detect malicious intent. Instead, it serves as an auxiliary statistical tool to estimate the behavioral distribution within each observation window and to instantiate period-level parameters. In addition to α , the trust evaluation model involves weight factors β , γ , and μ , which can be configured according to the service context. For instance, real-time messaging services may emphasize responsiveness (larger γ or μ), while highly interactive services may emphasize interaction continuity (larger β).

Let $\mathcal{O} = \{v \in V \mid \text{label}(v) = -1\}$ denote the set of outliers identified by DBSCAN within a given time window. We define the heterogeneity index h as:

$$h = \frac{|\mathcal{O}|}{|V|} \quad (11)$$

The weight adjustment coefficient α is then computed as:

$$\alpha = \text{clip}(1 - h, 0, 1) \quad (12)$$

Intuitively, a higher proportion of outliers signifies increased behavioral heterogeneity within the system. Under such conditions, a more conservative decay regime is enforced to mitigate potential risks.

To distinguish heterogeneous behavioral patterns among accounts, we adopt DBSCAN, a density-based unsupervised clustering method, on the processed on-chain telemetry features. Specifically, we first compute the delay-related feature using the historical processing delay algorithm, and then apply DBSCAN on the $(\text{interval}, \text{delay})$ feature space to separate dense normal-behavior regions from sparse outliers. Fig. 5 visualizes the clustering results. The dense cluster near zero delay represents consistently honest nodes, whereas the sparse outliers in the negative region indicate malicious behavior. The proportion of these outliers defines the heterogeneity index h , which inversely adjusts α to accelerate trust decay under adversarial conditions.

Source code for the certificate contracts and the datasets used in our analysis are available at:

<https://github.com/Nielemola/BC-TRUST>.

VI. ANALYSIS AND PRACTICAL CONSIDERATIONS

A. Cold Start and Privacy Discussion

1) *Cold Start*: Cold start is unavoidable when a newly joined node has insufficient interaction evidence. BC-TRUST avoids assigning arbitrary initial trust values; instead, it delays trust aggregation until minimal evidence is available in the current window. In addition, binding on-chain identities to verifiable entities can provide limited prior evidence that constrains early trust evolution without relying purely on recommendation propagation.

2) *Privacy*: Ethereum accounts are pseudonymous rather than anonymous, and on-chain transaction calls are publicly observable. BC-TRUST does not claim full anonymity; it minimizes on-chain exposure by avoiding personal identifiers and service payload content and relies only on coarse-grained interaction metadata to compute behavioral signals.

B. Qualitative Discussion of Additional Threats

1) *Whitewashing Attacks (WA)*: To mitigate whitewashing attacks, where malicious nodes abandon low-reputation accounts to re-enter as new identities, BC-TRUST imposes significant re-entry costs. Since trust links are reciprocal, existing PNs will refuse to maintain links with declining SNs, while other PNs remain hesitant to onboard new accounts lacking a proven social contribution history. Consequently, a malicious PN can only host a new SN by either registering a new node, which leaves immutable, auditable traces on the blockchain or by sacrificing its own established reputation. Given the PN's strong ties to real-world identity, such reputation loss acts as a powerful deterrent, ensuring accountability within the SIoT framework.

While BC-TRUST leverages transaction fees and external authentication to deter whitewashing and Sybil attacks, we acknowledge that this defense is context-dependent. In environments characterized by negligible transaction fees or weak identity vetting, attackers could theoretically maintain numerous realistic-looking identities at a minimal cost. To preserve robustness in such low-friction environments, BC-TRUST would need to integrate supplementary Sybil-resistance mechanisms. Practical extensions include staking mechanisms, where service nodes must lock a financial deposit in the smart contract that is slashed upon sustained malicious behavior; admission control governed by decentralized autonomous organizations (DAOs) to vet new entrants; or tighter coupling with external identity anchors, such as Decentralized Identifiers (DIDs) or enterprise Public Key Infrastructure (PKI), ensuring that every on-chain identity strictly corresponds to a verified physical entity.

2) *Self-Promoting Attacks (SPA)*: By recommending their positive activity to other nodes in the network, malicious nodes can conceal their malicious activity and become service providers. The proposed model provides resistance against

self-promotion attacks by tying trust value to the objective behavior of nodes on the blockchain over a given timeframe. Because the blockchain is tamper-proof and auditable, it is a dependable source of trust in this model.

3) *DDoS*: A DDoS attack can overload multiple nodes with a high volume of virtual requests simultaneously, resulting in the affected nodes being unable to function normally, causing system congestion. In the proposed model, the blockchain generates and tallies the interactions. While blockchain decentralization does not inherently prevent denial-of-service attacks, the proposed model avoids centralized trust computation points and does not rely on on-chain aggregation. As a result, adversarial traffic primarily increases economic cost through gas consumption rather than directly compromising trust evaluation integrity.

C. Semi-Formal Analysis of Trust Bounds and Stability

To theoretically justify the empirical results in Section VII, we provide a semi-formal analysis bounding the system's trust distortion.

Proposition 1 (Conditions for Honest Node Stability): Let p be the proportion of malicious nodes. If $p < 0.5$, an honest node h will reliably stay within the tolerance band $|S_w(h)| \leq \tau$.

The deviation score $S_w(x)$ utilizes $\text{median}(TSN_w)$ and $\text{MAD}(TSN_w)$ as baseline estimators. In robust statistics, both estimators have a breakdown point of 0.5. Therefore, as long as malicious nodes do not hold a strict majority ($p < 0.5$), the baseline is anchored entirely to the honest distribution TSN_H . The numerator $TSN_w(h) - \text{median}(TSN_w)$ reflects only natural variance, ensuring the standardized score is mathematically bounded within τ , independent of extreme adversarial outliers.

Proposition 2 (Bounds on Trust Distortion under Attacks): Under recommendation manipulation (e.g., ballot-stuffing), an attacker's trust inflation is strictly bounded by its direct behavioral evidence.

The aggregated score is a convex combination $TSN(x) = \lambda \times DT + (1 - \lambda) \times RT$. Even if colluding Sybils inflate the recommended trust to its maximum limit ($RT_{\max}$), malicious physical delays strictly decrease DT . Consequently, the attacker's trust is mathematically capped by the structural bound $\sup(TSN) \leq \lambda \times DT_{\text{degraded}} + (1 - \lambda) \times RT_{\max}$. Thus, infinite trust inflation is impossible without maintaining honest direct interactions.

VII. EVALUATION

We evaluate BC-TRUST via simulation-based analysis to examine whether trust evolution aligns with the model's design objectives under representative adversarial patterns in an Ethereum-based SIoT setting. We do not claim exact attack detection; instead, we focus on three properties: bounded evolution under non-adversarial patterns, stable window-to-window behavior without divergence, and resistance to sustained distortion when interaction evidence is strategically manipulated. Unless stated otherwise, nodes with fewer than three interactions in a window are treated as having insufficient evidence and are excluded from trust aggregation in that window.

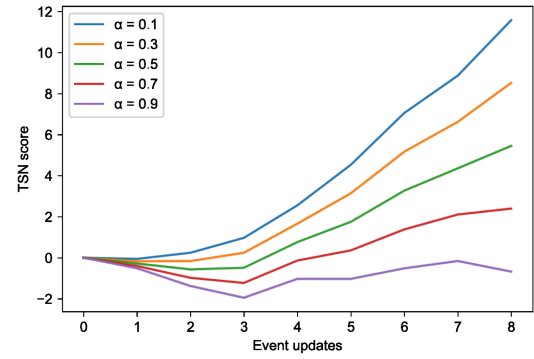


Fig. 6. Trust evolution under different telemetry-homogeneity contexts (without time decay).

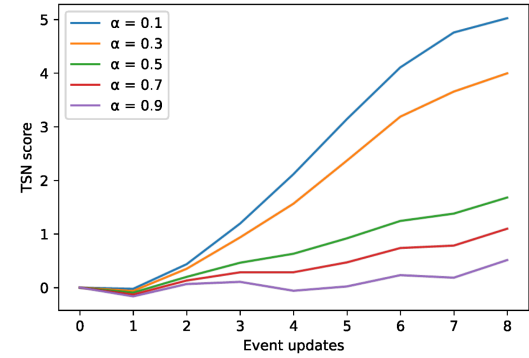


Fig. 7. Trust evolution under different telemetry-homogeneity contexts (with time decay).

A. Context Study and Decay-Regime Sensitivity

This subsection examines how the decay-regime switch controlled by α shapes trust evolution at the window level. We consider two telemetry-homogeneity context states: $\alpha = 0.7$ and $\alpha = 0.3$. Figs. 6 and 7 compare trust trajectories with and without the time-decay function. When $\alpha < 0.7$, trust may increase faster from recent evidence but is constrained by stronger decay, leading to an earlier plateau. When $\alpha \geq 0.7$, trust evolves more conservatively and preserves longer temporal memory due to slower decay. The regime switch follows (4) and Fig. 2.

As shown in Figs. 6 and 7, the overall shape of the trust trajectories appears uniform across different α values. This is deliberately designed by keeping the underlying sequence of behavioral evidence strictly identical for all test cases to isolate the impact of the decay parameter. The identical underlying events dictate the uniform trend of the curves, while the varying α values dictate the magnitude of the final TSN scores. A smaller α imposes a stricter temporal decay, significantly flattening the curve and limiting trust accumulation compared to a larger α .

B. Trust Evolution Under Adversarial Interaction Patterns

We simulate representative behavioral patterns corresponding to some attacks, focusing on how such patterns influence trust evolution rather than on precise attack attribution, and the attack resistance of the proposed model below.

1) *Malicious Attacks(MA)*: In MA, adversarial-pattern nodes manipulate recommendation values to contradict the

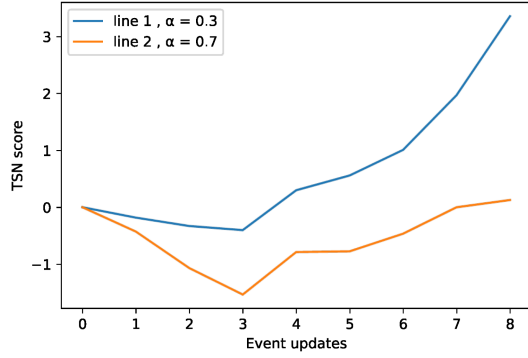


Fig. 8. Malicious comments on normal behavior.

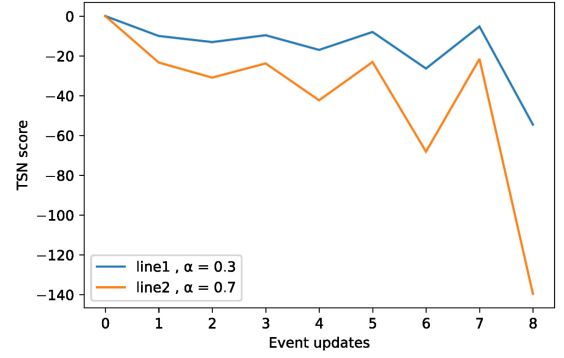


Fig. 10. Trust evolution under on-off behavior patterns (OA).

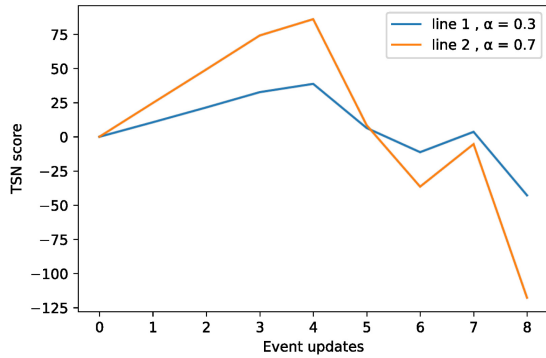


Fig. 9. Positive comments on malicious behavior.

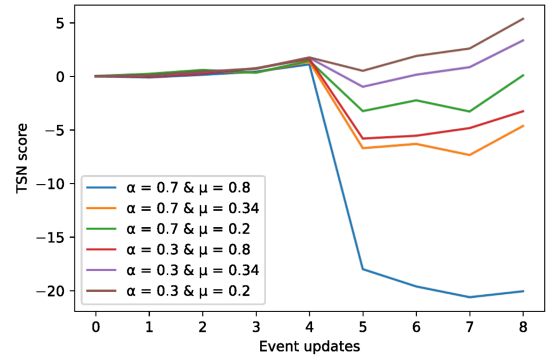


Fig. 11. Trust evolution under discriminatory interaction patterns (DA).

target node's direct behavioral evidence. As shown in Fig. 8. To isolate the effect of recommendation manipulation, we keep the direct-trust evidence in a controlled regime. Specifically, we set IA , PD , and RD to values close to the neutral baseline, centering near zero-axis following normalization, so that the direct-trust component remains stable across windows, and we set $\beta = \gamma = \mu$ for simplicity. We then evaluate two representative cases: *bad-mouthing*, where an attacker assigns low recommendation scores to a target whose direct-trust evidence remains benign-like (Fig. 8); and *ballot-stuffing*, where colluding nodes assign high recommendation scores to a target whose direct-trust evidence indicates degraded behavior (Fig. 9). This setup highlights how BC-TRUST balances direct evidence and recommendations under strategic recommendation manipulation. We have designated SN as a malicious node, with its direct trust value indicating a significantly low state. Meanwhile, the other malicious nodes on the network have evaluated it positively.

Fig. 8 shows that malicious evaluations can temporarily depress honest nodes' trust, but windowed direct evidence drives recovery toward baseline. Under heavy malicious presence (Line 1), honest nodes may exhibit short-lived trust inflation from contradictory feedback; however, this effect is not sustainable and fades as subsequent windows accumulate direct evidence. In Fig. 9, malicious nodes receiving inflated evaluations gain only marginal, transient trust before a rapid decline, indicating that BC-TRUST emphasizes persistent behavior over momentary feedback. Irregular and sparse malicious interaction

patterns further prevent attackers from sustaining high trust across windows.

2) *On-Off Attacks (OA)*: Malicious nodes often employ on-off attacks, strategically accumulating trust through honest behavior only to exploit the system once a reputation threshold is met. To simulate this, we staggered the values of IA , PD , and RD between extreme intervals to emulate behavioral switching. To capture the simultaneous impact on these metrics, the weighting parameters β , γ , and μ were set equally, while recommendation trust (RT) was fixed at 3 to isolate the effects on the Transaction Trust Score (TSN). As shown in Fig. 10, we further modulated the heterogeneity index α . Results indicate that while the malicious node's trust value fluctuates in alignment with the attack cycles, the aggregate trust trajectory exhibits a significant downward trend. This confirms that BC-TRUST prevents sustained trust inflation; short-term manipulations are effectively constrained, forcing the trust value to converge toward a regime dictated by the node's long-term on-chain evidence.

3) *Discriminatory Attacks (DA)*: RD captures function-selective delay. As shown in Fig. 11, by varying α and μ , we evaluate the sensitivity of trust trajectories to discriminatory interaction patterns. The results indicate that, when $\alpha = 0.7$, a larger μ amplifies the impact of RD and leads to clearer trust separation between discriminatory and non-adversarial patterns. When $\alpha = 0.3$, increasing μ changes the contribution of RD , but the overall trajectory shift is smaller than in the $\alpha = 0.7$ context because trust evolution is dominated by the more aggressive decay regime in this setting. However, as the figure demonstrates,

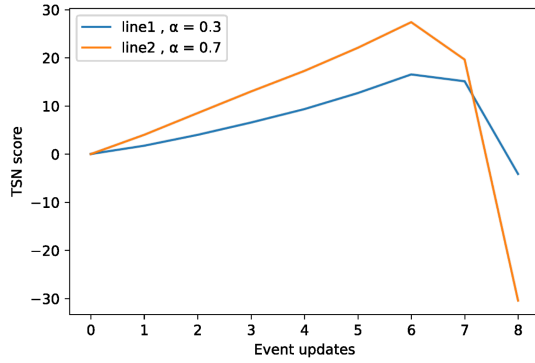


Fig. 12. Trust evolution under Sybil-like interaction patterns (SA).

when α is smaller, normal nodes show higher trust values due to their normal behavior.

4) *Sybil Attacks (SA)*: Rather than relying on cryptographic prevention, the proposed model disincentivizes Sybil behavior by elevating the economic and behavioral costs of maintaining consistent interaction patterns across multiple identities. Although Ethereum permits permissionless account generation, impersonating an existing account remains computationally infeasible without the corresponding private key or mnemonic. To counter self-promotion and recommendation manipulation by malicious PNs, BC-TRUST leverages the IA signal. As illustrated in Fig. 12, by tuning the sensitivity parameter β to emphasize IA, the model effectively constrains long-term trust inflation. For a Sybil node to provide sustainable trust evidence, it must maintain a high-frequency interaction pattern, which incurs prohibitive gas costs on the Ethereum network. Consequently, the proposed signals transform the pursuit of artificial trust into a cost-ineffective endeavor, thereby suppressing large-scale Sybil attacks through economic deterrence.

Overall, these scenarios show that trust trajectories remain differentiated across contexts, and short-term manipulation does not translate into sustained trust inflation over successive windows. Under limited interaction intervals, some adversarial patterns are not sufficiently expressed within a single window, which reduces the observable deviation in delay-related signals. Trust trajectories nevertheless remain bounded under mixed patterns.

C. Efficiency and Stress Test

Trust can be recomputed by any party from on-chain logs; in our prototype we perform off-chain replay for scalability measurement. We additionally run a stress test by generating 100 nodes and synthesizing windowed IA/PD/RD/RT signals for multiple update rounds. Fig. 13 shows the resulting trust trajectories under mixed patterns, illustrating that the windowed computation remains bounded at this scale.

Fig. 14 compares our approach against simpler heuristics. Fig. 14(a) compares our polynomial integration against a Simple Moving Average baseline under bursty SIoT traffic. While sudden interaction bursts cause extreme volatility in the moving average, our polynomial integration acts as a low-pass filter to smooth high-frequency jitter and provide a stable evaluation of

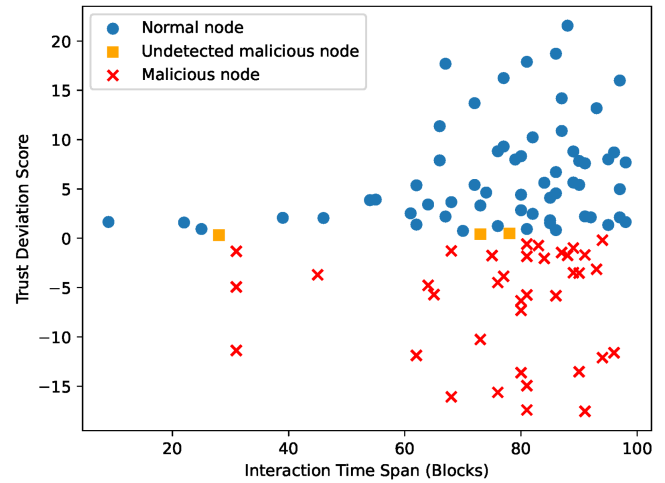


Fig. 13. Stress-test results.

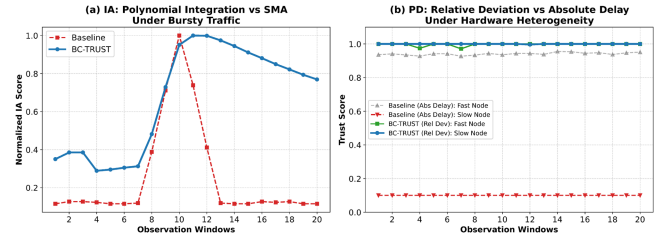


Fig. 14. Baseline comparisons justifying algorithmic complexity. (a) IA evaluation under bursty traffic. (b) PD evaluation in heterogeneous networks.

interaction willingness. Fig. 14(b) demonstrates the necessity of measuring relative deviation instead of absolute delay under hardware heterogeneity. When simulating a high-performance node with 15 ms delay and a resource-constrained node with 300 ms delay, using absolute delay unfairly misclassifies the constrained node as malicious with a score near 0.1. Through recursive relative deviation, BC-TRUST successfully isolates malicious degradation from hardware limits to assign fair and stable trust scores to all honest nodes.

D. Comparison

Baselines: We first provide a qualitative comparison with four recent trust-management schemes that represent complementary design philosophies, including lightweight blockchain-based trust for resource-constrained IoT [20], decentralized dynamic trust for IoMT [21], learning-based trust inference [26], and signed-network-embedding-based trust modeling [29]. Table II summarizes their differences in update granularity, trust computation, robustness coverage, and privacy-related considerations.

Unified settings for quantitative comparison: To avoid unfair comparisons caused by heterogeneous datasets and threat models across prior works, we conduct a minimal unified-settings quantitative comparison against the most relevant baseline [21], which features a trust update structure most similar to ours—specifically one integrating direct and recommendation trust under time-varying behaviors. Both methods consume the same windowed traces and attack scripts, and we report quantitative robustness and stability metrics in Table III.

TABLE II
QUALITATIVE COMPARISON WITH RECENT TRUST-MANAGEMENT BASELINES

Schemes	Scenarios evaluated	Trust updates	Trust calculations	Internal comparability	Privacy
Scheme [20]	Random / service attacks	Time	Blockchain-assisted reputation	×	×
Scheme [21]	Bad-mouthing; on-off	Time	Direct and recommended trust	×	×
Scheme [26]	Learning-based dynamic trust	Time	Deep learning inference	×	×
Scheme [29]	Self-promoting; bad-mouthing	Event	Signed-network embeddings	×	×
BC-TRUST	Bad-mouthing; ballot-stuffing; on-off	Time and Event	Global trust	✓	✓

TABLE III
UNIFIED-SETTINGS QUANTITATIVE COMPARISON

Attack	Metric	[21]	BC-TRUST
Bad-mouthing	Var(T) ↓	0.0036±0.0001	0.0029±0.0001
	Distortion ↓	0.0873±0.0031	0.0615±0.0028
	ASR ↓	0.0000±0.0000	0.0000±0.0000
Ballot-stuffing	Var(T) ↓	0.0039±0.0001	0.0035±0.0001
	Distortion ↓	0.0846±0.0025	0.0558±0.0021
	ASR ↓	0.0000±0.0000	0.0000±0.0000
On-off	Var(T) ↓	0.0039±0.0001	0.0035±0.0001
	Distortion ↓	0.1426±0.0002	0.1061±0.0001
	ASR ↓	0.7100±0.0686	0.6833±0.2607

Metrics: We report stability as the window-to-window variance $\text{Var}(T)$ for benign-like behavior, robustness as sustained trust distortion and attack success rate (ASR), and overhead, encompassing runtime per window as well as the on-chain footprint measured in bytes and transactions per window. Since the standardized deviation score S is real-valued and zero-centered, a node is categorized as benign if its tail-mean deviation remains within a predefined tolerance band, i.e., $|\hat{S}_v| \leq \tau$. Here, $\hat{S}_v$ denotes the tail-mean of $S_w(v)$ calculated over a sliding observation window comprising the most recent 20% of the recorded time intervals.

As shown in Table III, BC-TRUST consistently improves stability and reduces sustained distortion under strategic recommendation manipulation and on-off behavior. The variance under on-off behavior is higher because the attack phase alignment across windows affects the tail-mean trust used by ASR, leading to seed-dependent fluctuations.

For bad-mouthing, BC-TRUST reduces $\text{Var}(T)$ from 0.0036 to 0.0029 and reduces distortion from 0.0873 to 0.0615. For ballot-stuffing, BC-TRUST reduces $\text{Var}(T)$ from 0.0039 to 0.0035 and reduces distortion from 0.0846 to 0.0558. Under on-off behavior, BC-TRUST reduces distortion from 0.1426 to 0.1061, while achieving a slightly lower ASR on average. These results indicate that, under unified traces, BC-TRUST better constrains window-wise trust drift induced by strategic recommendation manipulation and temporal inconsistency.

All results in Table III are averaged over $R = 10$ random seeds with $N = 100$ nodes and malicious ratio $p = 0.3$. We use a fixed band width τ for all methods under the same unified setting, and compute $\hat{S}_v$ as the tail-mean over the last 20% windows.

VIII. CONCLUSION

This paper presented BC-TRUST, a blockchain-assisted trust management system for SIoT environments. BC-TRUST computes trust from time-sensitive behavioral evidence recorded on-chain, enabling transparent and auditable assessment among heterogeneous devices. Rather than relying on predefined

initialization or purely recommendation-driven propagation, the model emphasizes behavioral consistency over time and supports practical trust bootstrapping when prior information is scarce. The system is designed for deployment in private and federated SIoT settings and does not require any modification to blockchain consensus. Experiments show that BC-TRUST achieves stable trust evolution and effectively limits sustained trust distortion under representative adversarial interaction patterns, without requiring explicit attack attribution or node-level conviction. Furthermore, because the trust evaluation relies on objective protocol telemetry and off-chain computation rather than application-specific semantics, the model is highly scalable and transferable to other distributed social networks. Future work will investigate access-control integration for service coordination, adaptive parameter tuning, and stronger privacy protection via lightweight cryptographic mechanisms.

REFERENCES

- [1] J. S. Yalli, M. H. Hasan, and A. A. Badawi, "Internet of Things (IoT): Origins, embedded technologies, smart applications, and its growth in the last decade," *IEEE Access*, vol. 12, pp. 91357–91382, 2024.
- [2] H. I. Sarker, A. I. Khan, Y. B. Abushark, and F. Alsolami, "Internet of Things (IoT) security intelligence: A comprehensive overview, machine learning solutions and research directions," *Mobile Netw. Appl.*, vol. 28, no. 1, pp. 296–312, 2023.
- [3] S. Sagar et al., "Understanding the trustworthiness management in the social Internet of Things: A survey," *Comput. Netw.*, vol. 251, 2024, Art. no. 110611.
- [4] M. M. Rad, A. M. Rahmani, A. Sahafi, and N. N. Qader, "Social Internet of Things: Vision, challenges, and trends," *Hum. Centric Comput. Inf. Sci.*, vol. 10, 2020, Art. no. 52.
- [5] E. H. Houssein, M. A. Othman, W. M. Mohamed, and M. Younan, "Internet of things in smart cities: Comprehensive review, open issues, and challenges," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34941–34952, Nov. 2024.
- [6] W. Z. Khan, Q.-U.-A. Arshad, S. Hakak, M. K. Khan, "Trust management in social Internet of Things: Architectures, recent advancements, and future challenges," *IEEE Internet Things J.*, vol. 8, no. 10, pp. 7768–7788, May 2021.
- [7] W. W. Smari, P. Clemente, and J.-F. Lalande, "An extended attribute based access control model with trust and privacy: Application to a collaborative crisis management system," *Future Gener. Comput. Syst.*, vol. 31, pp. 147–168, 2014.
- [8] G. D. Putra, V. Dedeoglu, S. S. Kanhere, and R. Jurdak, "Trust management in decentralized IoT access control system," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency*, 2020, pp. 1–9.
- [9] C. Wang, G. Liu, and T. Jiang, "Malicious node detection in wireless weak-link sensor networks using dynamic trust management," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 12866–12877, Dec. 2024.
- [10] A. Mahmood, Q. Z. Sheng, W. E. Sheng, Y. Wang, and S. Sagar, "Toward a distributed trust management system for misbehavior detection in the Internet of Vehicles," *ACM Trans. Cyber Phys. Syst.*, vol. 7, no. 3, pp. 16:1–16:25, 2023.
- [11] A. Papenmeier, D. Kern, G. Engleblenne, and C. Seifert, "It's complicated: The relationship between user trust, model accuracy and explanations in AI," *ACM Trans. Comput. Hum. Interact.*, vol. 29, no. 4, pp. 35:1–35:33, 2022.

- [12] S. Mehrotra, C. Deguchi, O. Vereschak, C. M. Jonker, and M. L. Tielman, "A systematic review on fostering appropriate trust in human-AI interaction: Trends, opportunities and challenges," *ACM J. Responsible Comput.*, vol. 1, no. 4, pp. 1–45, 2024.
- [13] H. Xia, F. Xiao, S. Zhang, X. Cheng, and Z. Pan, "A reputation-based model for trust evaluation in social cyber-physical systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 2, pp. 792–804, 2nd Quart. 2020.
- [14] L. Atzori, A. Iera, and G. Morabito, "IIoT: Giving a social structure to the Internet of Things," *IEEE Commun. Lett.*, vol. 15, no. 11, pp. 1193–1195, Nov. 2011.
- [15] L. Wei, Y. Yang, J. Wu, C. Long, and B. Li, "Trust management for Internet of Things: A comprehensive study," *IEEE Internet Things J.*, vol. 9, no. 10, pp. 7664–7679, May 2022.
- [16] N. Truong, G. M. Lee, K. Sun, F. Guitton, and Y. Guo, "A blockchain-based trust system for decentralised applications: When trustless needs trust," *Future Gener. Comput. Syst.*, vol. 124, pp. 68–79, 2021.
- [17] H. Yang, J. Yuan, H. Yao, Q. Yao, A. Yu, and J. Zhang, "Blockchain-based hierarchical trust networking for jointcloud," *IEEE Internet Things J.*, vol. 7, no. 3, pp. 1667–1677, Mar. 2020.
- [18] W. She, Q. Liu, Z. Tian, J.-S. Chen, B. Wang, and W. Liu, "Blockchain trust model for malicious node detection in wireless sensor networks," *IEEE Access*, vol. 7, pp. 38947–38956, 2019.
- [19] S. Malik, V. Dedeglu, S. S. Kanhere, and R. Jurdak, "TrustChain: Trust management in blockchain and IIoT supported supply chains," in *Proc. IEEE Int. Conf. Blockchain*, 2019, pp. 184–193.
- [20] M. Deng et al., "Lightweight trust management scheme based on blockchain in resource-constrained intelligent IIoT systems," *IEEE Internet Things J.*, vol. 11, no. 15, pp. 25706–25719, Aug. 2024.
- [21] X. Xiang, J. Cao, W. Fan, S. Xiang, and G. Wang, "Blockchain enabled dynamic trust management method for the internet of medical things," *Decis. Support Syst.*, vol. 180, 2024, Art. no. 114184.
- [22] S. K. Awasthi and Y. N. Singh, "AbsoluteTrust: Algorithm for aggregation of trust in peer-to-peer networks," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 176–189, Jan./Feb. 2022.
- [23] A. A. Adewuyi, H. Cheng, Q. Shi, J. Cao, A. MacDermott, and X. Wang, "CTRUST: A dynamic trust model for collaborative applications in the Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 5432–5445, Jun. 2019.
- [24] C. Fernández-Gago, F. Moyano, and J. López, "Modelling trust dynamics in the Internet of Things," *Inf. Sci.*, vol. 396, pp. 72–82, 2017.
- [25] C. Boudagdigue, A. Benslimane, A. Kobbane, and J. Liu, "Trust management in industrial Internet of Things," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3667–3682, May 2020.
- [26] F. Ullah et al., "Deep trust: A novel framework for dynamic trust and reputation management in the Internet of Things (IIoT)-based networks," *IEEE Access*, vol. 12, pp. 87407–87419, 2024.
- [27] U. Jayasinghe, G. M. Lee, T.-W. Um, and Q. Shi, "Machine learning based trust computational model for IIoT services," *IEEE Trans. Sustain. Comput.*, vol. 4, no. 1, pp. 39–52, 1st Quart. 2019.
- [28] I.-R. Chen, F. Bao, and J. Guo, "Trust-based service management for social Internet of Things systems," *IEEE Trans. Dependable Secure Comput.*, vol. 13, no. 6, pp. 684–696, Nov./Dec. 2016.
- [29] S. Dhelim, N. Aung, M. T. Kechadi, H. Ning, L. Chen, and A. Lakas, "Trust2Vec: Large-scale IIoT trust management system based on signed network embeddings," *IEEE Internet Things J.*, vol. 10, no. 1, pp. 553–562, Jan. 2023.
- [30] K. A. Awan, I. Ud Din, A. Almogren, M. Guizani, A. Altameem, and S. U. Jadoon, "RobustTrust – A pro-privacy robust distributed trust management mechanism for Internet of Things," *IEEE Access*, vol. 7, pp. 62095–62106, 2019.
- [31] D. E. Kouicem, Y. Imine, A. Bouabdallah, and H. Lakhlef, "Decentralized blockchain-based trust management protocol for the Internet of Things," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1292–1306, Mar./Apr. 2022.
- [32] A. Sharma, E. S. Pilli, A. P. Mazumdar, and P. Gera, "Towards trustworthy Internet of Things: A survey on trust management applications and schemes," *Comput. Commun.*, vol. 160, pp. 475–493, 2020.
- [33] D. Gambetta et al., "Can we trust trust," *Trust: Mak. Breaking Cooperative Relations*, vol. 13, pp. 213–237, 2000.
- [34] F. W. McElroy, "A necessary and sufficient condition that ordinary least-squares estimators be best linear unbiased," *J. Amer. Stat. Assoc.*, vol. 62, no. 320, pp. 1302–1304, 1967.
- [35] PolyFit, "Introduction to the polyfit function in MATLAB," 2023. [Online]. Available: <https://ww2.mathworks.cn/help/matlab/ref/polyfit.html>
- [36] Remix, "Remix IDE for ethereum smart contract programming," 2023. [Online]. Available: <https://remix.ethereum.org/>
- [37] Ganache, "Ganache one click blockchain," 2023. [Online]. Available: <https://trufflesuite.com/ganache/>
- [38] Web3.py, "Web3 python API to interact with ethereum nodes," 2023. [Online]. Available: <https://web3py.readthedocs.io/en/stable/>

Song Luo received the PhD degree in computer software and theory from the Institute of Software, School of Electronics Engineering and Computer Science, Peking University, China, in 2011. He is presently working as an associate professor with the College of Computer Science and Engineering, Chongqing University of Technology, Chongqing, China. His research interests include public key cryptography, access control, and network security.

Tan Hu received the MS degree from the Chongqing University of Technology. He is currently working toward the PhD degree with the School of Cyber Engineering, Xidian University, Xi'an, China. His research interests include network security, access control, trust management, and blockchain.

Cong Sun (Member, IEEE) received the BS degree in computer science from Zhejiang University, in 2005, and the PhD degree in computer science from Peking University, in 2011. He is a full professor with the School of Cyber Engineering and the State Key Laboratory of Integrated Services Networks, Xidian University. His research interests include software security, and program analysis.

Zhi Guan is an associate professor with Software Engineering Research Center of Peking University. His recent research interests include blockchain security and cryptography applications in blockchain. He has published more than 30 papers in the above areas and his research is supported by two National Key R&D Programs. He is also the author and maintainer of the GmSSL crypto library.